6 JDK concurrency libraries

This chapter covers

- Atomic classes
- Locks classes
- Concurrent data structures
- BlockingQueues
- Futures and CompletableFuture
- Executors

In this chapter we'll cover what every well-grounded developer should know about `java.util.concurrent` and how to use the toolbox of concurrency building blocks it provides. The aim is that by the end of the chapter, you'll be ready to start applying these libraries and concurrency techniques in your own code.

6.1 Building blocks for modern concurrent applications

As we saw in the previous chapter, Java has supported concurrency since the very beginning. However, with the advent of Java 5 (which was itself over 15 years ago), a new way of thinking about concurrency in Java emerged. This was spearheaded by the package `java.util.concurrent`, which contained a rich new toolbox for working with multithreaded code.

NOTE This toolbox has been enhanced with subsequent versions of Java, but the classes and packages that were introduced with Java 5 still work the same way, and they're still very valuable to the working developer.

If you (still!) have existing multithreaded code that is based solely on the older (pre-Java 5) approaches, you should consider refactoring it to use `java.util.concurrent`. In our experience, your code will be improved if you make a conscious effort to port it to the newer APIs—the greater clarity and reliability will be well worth the effort expended to migrate in almost all cases.

We're going to take a tour through some of the headline classes in `java.util .concurrent` and related packages, such as the atomic and locks packages. We'll get you started using the classes and look at examples of use cases for them.

You should also read the Javadoc for them and try to build up your familiarity with the packages as a whole. Most developers find that the higher level of abstraction that they provide makes concurrent programming much easier.

6.2 Atomic classes

The package `java.util.concurrent.atomic` contains several classes that have names starting with `Atomic`, for example, `AtomicBoolean`, `AtomicInteger`, `AtomicLong`, and `AtomicReference`. These classes are one of the simplest examples of a *concurrency primitive*—a class that can be used to build workable, safe concurrent applications.

WARNING Atomic classes don't inherit from the similarly named classes, so `AtomicBoolean` can't be used in place of a `Boolean`, and an `AtomicInteger` isn't an `Integer` (but it does extend `Number`).

The point of an atomic is to provide thread-safe mutable variables. Each of the four classes provides access to a single variable of the appropriate type.

NOTE The implementations of the atomics are written to take advantage of modern processor features, so they can be nonblocking (lock-free) if support is available from the hardware and OS, which it will be for virtually all modern systems.

The access provided is lock-free on almost all modern hardware, so the atomics behave in a similar way to a volatile field. However, they are wrapped in a Class API that goes further than what's possible with volatiles. This API includes atomic (meaning all-or-nothing) methods for suitable operations—including state-dependent updates (which are impossible to do with volatile variables without using a lock). The end result is that atomics can be a very simple way for a developer to avoid race conditions on shared data.

NOTE If you're curious as to how atomics are implemented, we will discuss the details in chapter 17, when we talk about internals and the class `sun.misc .Unsafe`.

A common use case for atomics is to implement something similar to sequence numbers, as you might find provided by an SQL database. This capability is accessed by using methods such as the atomic `getAndIncrement()` on the `AtomicInteger` or `AtomicLong`

classes. Let's look at how we would rewrite the `Account` example from chapter 5 to use an atomic:

```
    private static AtomicInteger nextAccountId = new AtomicInteger(1);

    private final int accountId;
    private double balance;

    public Account(int openingBalance) {
       balance = openingBalance;
       accountId = nextAccountId.getAndIncrement();
    }
```

As each object is created, we make a call to `getAndIncrement()` on the static instance of `AtomicInteger`, which returns us an int value and atomically increments the mutable variable. This atomicity guarantees that it is impossible for two objects to share the same `accountId`, which is exactly the property that we want (just like a database sequence number).

NOTE We could add the `final` qualifier to the atomic, but it's not necessary because the field is `static` and the class doesn't provide any way to mutate the field.

For another example, here is how we would rewrite our volatile shutdown example to use an `AtomicBoolean`:

```
public class TaskManager implements Runnable {
    private final AtomicBoolean shutdown = new AtomicBoolean(false);

    public void shutdown() {
       shutdown.set(true);
    }

    @Override
    public void run() {
       while (!shutdown.get()) {
          // do some work - e.g. process a work unit
       }
    }
}
```

As well as these examples, the `AtomicReference` is also used to implement atomic changes but to objects. The general pattern is that some modified (possibly immutable) state is built optimistically and can then be "swapped in" by using a *Compare-and-Swap* (CAS) operation on an `AtomicReference`.

Next, let's examine how `java.util.concurrent` models the core of the classic synchronization approach—the `Lock` interface.

6.3 Lock classes

The block-structured approach to synchronization is based on a simple notion of what a lock is. This approach has a number of shortcomings, as follows:

- Only one type of lock exists.
- It applies equally to all synchronized operations on the locked object.
- The lock is acquired at the start of the synchronized block or method.
- The lock is released at the end of the block or method.
- Either the lock is acquired or the thread blocks indefinitely—no other outcomes are possible.

If we were going to reengineer the support for locks, we could potentially change several things for the better:

- Add different types of locks (such as reader/writer locks).
- Not restrict locks to blocks (allow a lock in one method and an unlock in another).
- If a thread cannot acquire a lock (e.g., if another thread has the lock), allow the thread to back out or carry on or do something else—a `tryLock()`.
- Allow a thread to attempt to acquire a lock and give up after a certain amount of time.

The key to realizing all of these possibilities is the `Lock` interface in `java.util.concurrent.locks`. This interface ships with the following implementations:

- `ReentrantLock`—This is essentially the equivalent of the familiar lock used in Java synchronized blocks but more flexible.
- `ReentrantReadWriteLock`—This can provide better performance in cases where there are many readers but few writers.

NOTE Other implementations exist, both within the JDK and written by third parties, but these are by far the most common.

The `Lock` interface can be used to completely replicate any functionality that is offered by block-structured concurrency. For example, listing 6.1 shows the example from chapter 5 for how to avoid deadlock rewritten to use `ReentrantLock`.

We need to add a lock object as a field to the class, because we will no longer be relying on the intrinsic lock on the object. We also need to maintain the principle that locks are always acquired in the same order. In our example the simple protocol we maintain is that the lock on the object with the lowest account ID is acquired *first*.

Listing 6.1 Rewriting deadlock example to use `ReentrantLock`

```
private final Lock lock = new ReentrantLock();

public boolean transferTo(SafeAccount other, int amount) {
   // We also need code to check to see amount > 0, throw if not
   // ...

   if (accountId == other.getAccountId()) {
      // Can't transfer to your own account
      return false;
   }

   var firstLock = accountId < other.getAccountId() ?
         lock : other.lock;
   var secondLock = firstLock == lock ? other.lock : lock;

   firstLock.lock();              (1)
   try {
      secondLock.lock();          (2)
      try {
         if (balance >= amount) {
            balance = balance - amount;
            other.deposit(amount);
            return true;
         }
         return false;
      } finally {
         secondLock.unlock();
      }
   } finally {
      firstLock.unlock();
   }
}
```

❶ The firstLock object has a lower account ID.

❷ The secondLock object has a higher account ID.

The pattern of an initial call to `lock()` combined with a `try ... finally` block, where the lock is released in the `finally`, is a great addition to your toolbox.

NOTE The locks, like much of `java.util.concurrent`, rely on a class called `AbstractQueuedSynchronizer` to implement their functionality.

The pattern works very well if you're replicating a situation that is similar to one where you'd have used block-structured concurrency. On the other hand, if you need to pass around the `Lock` objects (such as by returning it from a method), you can't use this pattern.

13.1 Condition objects

Another aspect of the API provided by `java.util.concurrent` are the *condition objects*. These objects play the same role in the API as `wait()` and `notify()` do in the original intrinsic API but are more flexible. They provide the ability for threads to wait indefinitely for some condition and to be woken up when that condition becomes true.

However, unlike the intrinsic API (where the object monitor has only a single condition for signaling), the `Lock` interface allows the programmer to create as many condition objects as they like. This allows a separation of concerns—for example, the lock can have multiple, disjoint groups of methods that can use separate conditions.

A condition object (which implements the interface `Condition`) is created by calling the `newCondition()` method on a lock object (one that implements the `Lock` interface). As well as condition objects, the API provides a number of *latches* and *barriers* as concurrency primitives that may be useful in some circumstances.

6.4 CountDownLatch

The `CountDownLatch` is a simple concurrency primitive that provides a *consensus barrier*—it allows for multiple threads to reach a coordination point and wait until the barrier is released. This is achieved by providing an int value (the `count`) when constructing a new instance of `CountDownLatch`. After that point, two methods are used to control the latch: `countDown()` and `await()`. The former reduces the count by 1, and the latter causes the calling thread to block until the count reaches 0 (it does nothing if the count

is already 0 or less). In the following listing, the latch is used by each `Runnable` to indicate when it has completed its assigned work.

Listing 6.2 Using latches to signal between threads

```
public static class Counter implements Runnable {
   private final CountDownLatch latch;
   private final int value;
   private final AtomicInteger count;

   public Counter(CountDownLatch l, int v, AtomicInteger c) {
      this.latch = l;
      this.value = v;
      this.count = c;
   }

   @Override
   public void run() {
      try {
         Thread.sleep(100);
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      }
      count.addAndGet(value);   ❶
      latch.countDown();        ❷
   }
}
```

❶ Updates the count value atomically

❷ Decrements the latch

Note that the `countDown()` method is nonblocking, so once the latch has been decremented, the thread running the `Counter` code will exit.

We also need some driver code, shown here (exceptions elided):

```
var latch = new CountDownLatch(5);
var count = new AtomicInteger();
for (int i = 0; i < 5; i = i + 1) {
   var r  = new Counter(latch, i, count);
   new Thread(r).start();
}
```

```
latch.await();
System.out.println("Total: "+ count.get());
```

In the code, the latch is set up with a quorum value (in figure 6.1, the value is 2). Next, the same number of threads are created and initialized, so that processing can begin. The main thread awaits the latch and blocks until it is released. Each worker thread will perform a sleep and then `countDown()` once it has finished. The main thread will not proceed until both of the threads have completed their processing. This situation is show in figure 6.1.

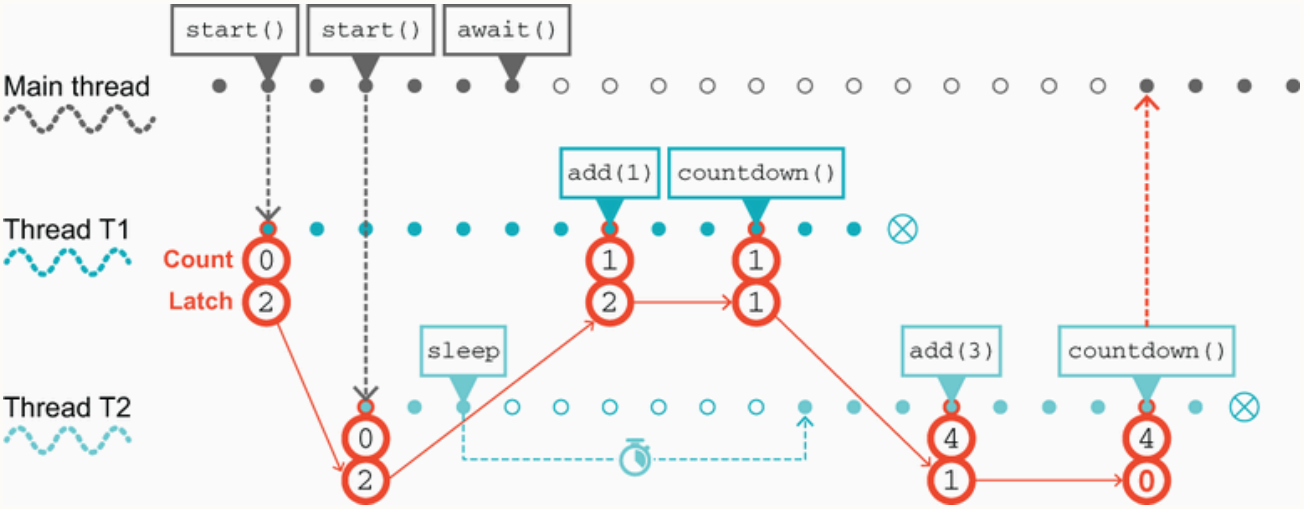


Figure 6.1 Using a `CountDownLatch`

To provide another example of a good use case for `CountDownLatch`, consider an application that needs to prepopulate several caches with reference data before the server is ready to receive incoming requests. We can easily achieve this by using a shared latch, a reference to which is held by each cache population thread.

When each cache finishes loading, the `Runnable` populating it counts down the latch and exits. When all the caches are loaded, the main thread (which has been awaiting the latch opening) can proceed and is ready to mark the service as up and begin handling requests.

The next class we'll discuss is one of the most useful classes in the multithreaded developer's toolkit: the `ConcurrentHashMap`.

6.5 ConcurrentHashMap

The `ConcurrentHashMap` class provides a concurrent version of the standard `HashMap`. In general, maps are a very useful (and common) data structure for building concurrent

applications. This is due, at least in part, to the shape of the underlying data structure. Let's take a closer look at the basic `HashMap` to understand why.

6.5.1 Understanding a simplified HashMap

As you can see from figure 6.2, the classic Java `HashMap` uses a function (the *hash function*) to determine which *bucket* it will store the key-value pair in. This is where the "hash" part of the class's name comes from.

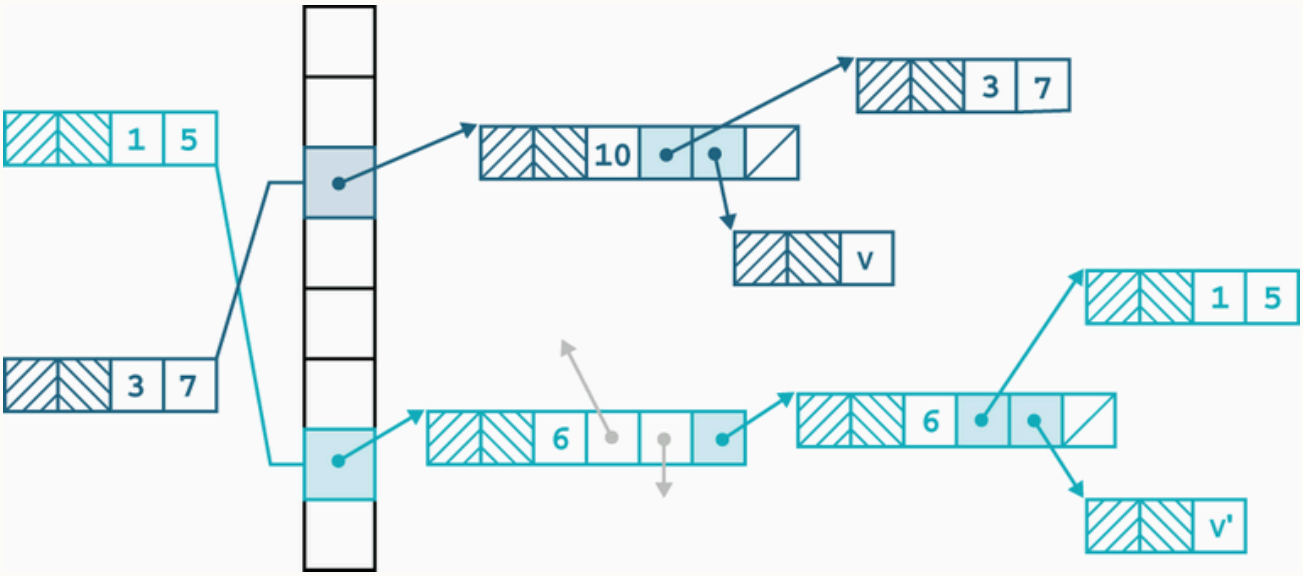


Figure 6.2 The classic view of a `HashMap`

The key-value pair is actually stored in a linked list (known as the *hash chain*) that starts from the bucket corresponding to the index obtained by hashing the key.

In the GitHub project that accompanies this book is a simplified implementation of a `Map<String, String>` —the `Dictionary` class. This class is actually based on the form of the `HashMap` that shipped as part of Java 7.

NOTE Modern Java versions ship a `HashMap` implementation that is significantly more complex, so in this explanation, we focus on a simpler version where the design concepts are more clearly visible.

The basic class has only two fields: the main data structure and the `size` field, which caches the size of the map for performance reasons, as shown next:

```
public class Dictionary implements Map<String, String> {
    private Node[] table = new Node[8];
    private int size;
```

```
@Override
public int size() {
    return size;
}

@Override
public boolean isEmpty() {
    return size == 0;
}
```

These rely on a helper class, called a `Node`, which represents a key-value pair and implements the interface `Map.Entry` as follows:

```
static class Node implements Map.Entry<String,String> {
    final int hash;
    final String key;
    String value;
    Node next;

    Node(int hash, String key, String value, Node next) {
        this.hash = hash;
        this.key = key;
        this.value = value;
        this.next = next;
    }

    public final String getKey()        { return key; }
    public final String getValue()      { return value; }
    public final String toString() { return key + "=" + value; }

    public final int hashCode() {
        return Objects.hashCode(key) ^ Objects.hashCode(value);
    }

    public final String setValue(String newValue) {
        String oldValue = value;
        value = newValue;
        return oldValue;
    }

    public final boolean equals(Object o) {
        if (o == this)
            return true;
        if (o instanceof Node) {
            Node e = (Node)o;
```

```
            if (Objects.equals(key, e.getKey()) &&
                Objects.equals(value, e.getValue()))
                return true;
        }
        return false;
    }
}
```

To look for a value in the map, we use the `get()` method, which relies on a couple of helper methods, `hash()` and `indexFor()` as follows:

```
@Override
public String get(Object key) {
    if (key == null)
        return null;
    int hash = hash(key);
    for (Node e = table[indexFor(hash, table.length)];
         e != null;
         e = e.next) {
        Object k = e.key;
        if (e.hash == hash && (k == key || key.equals(k)))
            return e.value;
    }
    return null;
}

static final int hash(Object key) {
    int h = key.hashCode();
    return h ^ (h >>> 16);          (1)
}

static int indexFor(int h, int length) {
    return h & (length - 1);        (2)
}
```

(1) A bitwise operation to make sure that the hash value is positive

(2) A bitwise operation to make sure that the index is within the size of the table

First, the `get()` method deals with the irritating null case. Following that, we use the key object's hash code to construct an index into the array `table`. An unwritten assumption says that the size of `table` is a power of two, so the operation of `indexFor()` is basically a modulo operation, which ensures that the return value is a valid index into `table`.

NOTE This is a classic example of a situation where a human mind can determine that an exception (in this case, `ArrayIndexOutOfBoundsException`) will never be thrown, but the compiler cannot.

Now that we have an index into `table`, we use it to select the relevant hash chain for our lookup operation. We start at the head and walk down the hash chain. At each step we evaluate whether we've found our key object, as shown next:

```
if (e.hash == hash && ((k = e.key) == key || key.equals(k)))
   return e.value;
```

If we have, then we return the corresponding value. We store keys and values as pairs (really as `Node` instances) to allow for this approach.

The `put()` method is somewhat similar to the previous code:

```
@Override
public String put(String key, String value) {
   if (key == null)
      return null;

   int hash = hash(key.hashCode());
   int i = indexFor(hash, table.length);
   for (Node e = table[i]; e != null; e = e.next) {
      Object k = e.key;
      if (e.hash == hash && (k == key || key.equals(k))) {
         String oldValue = e.value;
         e.value = value;
         return oldValue;
      }
   }

   Node e = table[i];
   table[i] = new Node(hash, key, value, e);

   return null;
}
```

This version of a hashed data structure is not 100% production quality, but it is intended to demonstrate the basic behavior and approach to the problem, so that the concurrent case can be understood.

5.2 Limitations of Dictionary

Before we proceed to the concurrent case, we should mention that some methods from `Map` are not supported by our toy implementation, `Dictionary`. Specifically, `putAll()`, `keySet()`, `values()`, or `entrySet()` (which need to be defined, because the class implements `Map`) will simply `throw new UnsupportedOperationException()`.

We do not support these methods purely and solely due to complexity. As we will see several times in the book, the Java Collections interfaces are large and feature-rich. This is good for the end user, because they have a lot of power, but it means an implementor must supply a lot more methods.

In particular, methods like `keySet()` require an implementation of `Map` to supply instances of `Set`, and this frequently results in needing to write an entire implementation of the `Set` interface as an inner class. That is too much extra complexity for our examples, so we just don't support those methods in our toy implementation.

NOTE As we will see later in the book, the monolithic, complex, imperative design of the Collections interfaces presents various problems when we start to think about functional programming in detail.

The simple `Dictionary` class works well, within its limitations. However, it does not guard against the following two scenarios:

- The need to resize `table` as the number of elements stored increases
- Defending against keys that implement a pathological form of `hashCode()`

The first of these is a serious limitation. A major point of a hashed data structure is to reduce the expected complexity operations down from `O(N)` to `O(log N)`, for example, for value retrieval. If the table is not resized as the number of elements held in the map increases, this complexity gain is lost. A real implementation would have to deal with the need to resize the table as the map grows.

5.3 Approaches to a concurrent Dictionary

As it stands, `Dictionary` is obviously not thread-safe. Consider two threads—one trying to delete a certain key and the other trying to update the value associated with it. Depending on the ordering of operations, it is entirely possible for both the deletion and the update to report that they succeeded when in fact only one of them did. To resolve this, we have two fairly obvious (if naïve) ways to make `Dictionary` (and, by extension, general Java `Map` implementations) concurrent.

First off is the fully synchronization approach, which we met in chapter 5. The punch line is not hard to predict: this approach is unfeasible for most practical systems due to performance overhead. However, it's worth a small diversion to look at how we might implement it.

We have two easy ways to achieve simple thread safety here. The first is to copy the `Dictionary` class—let's call it `ThreadSafeDictionary` and then make all of its methods `synchronized`. This works but involves a lot of duplicated, cut-and-paste code.

Alternatively, we can use a synchronized wrapper to provide *delegation*—aka forwarding—to an underlying object that actually houses the dictionary. Here's how we can do that:

```
public final class SynchronizedDictionary extends Dictionary {
    private final Dictionary d;

    private SynchronizedDictionary(Dictionary delegate) {
        d = delegate;
    }

    public static SynchronizedDictionary of(Dictionary delegate) {
        return new SynchronizedDictionary(delegate);
    }

    @Override
    public synchronized int size() {
        return d.size();
    }

    @Override
    public synchronized boolean isEmpty() {
        return d.isEmpty();
    }

    // ... other methods

}
```

This example has a number of problems, the most important of which is that the object `d` already exists and is not synchronized. This is setting ourselves up to fail—other code may modify `d` outside of a synchronized block or method, and we find ourselves in exactly the situation we discussed in the previous chapter. This is not the right approach for concurrent data structures.

We should mention that, in fact, the JDK provides just such an implementation—the `synchronizedMap()` method provided in the `Collections` class. It works about as well and is about as widely used, as you might expect.

A second approach is to appeal to immutability. As we will say, and say again, the Java Collections are large and complex interfaces. One way in which this manifests is that the assumption of mutability is baked throughout the collections. In no sense is it a separable concern that some implementations may choose, or not, to express—all implementations of `Map` and `List` must implement the mutating methods.

Due to this constriction, it might seem as though we have no way to model a data structure in Java that is both immutable and conforms to the Java Collections APIs—if it conforms to the APIs, the class must also provide an implementation of the mutation method. However, a deeply unsatisfactory back door exists. An implementation of an interface can always throw `UnsupportedOperationException` if it has not implemented a certain method. From a language design point of view, this is, of course, terrible. An interface contract should be exactly that—a contract.

Unfortunately, this mechanism and convention predates Java 8 (and the arrival of default methods) and thus represents an attempt to encode a difference between a "mandatory" method and an "optional" one, at a time when no such distinction actually existed in the Java language.

It is a bad mechanism and practice (especially because `UnsupportedOperationException` is a runtime exception), but we could use it something like this:

```
public final class ImmutableDictionary extends Dictionary {
    private final Dictionary d;

    private ImmutableDictionary(Dictionary delegate) {
        d = delegate;
    }

    public static ImmutableDictionary of(Dictionary delegate) {
        return new ImmutableDictionary(delegate);
    }

    @Override
    public int size() {
        return d.size();
    }
```

```
    @Override
    public String get(Object key) {
      return d.get(key);
    }

    @Override
    public String put(String key, String value) {
      throw new UnsupportedOperationException();
    }

    // other mutating methods also throw UnsupportedOperationException

  }
```

It can be argued that this is something of a violation of object-oriented principles—the expectation from the user is that this is a valid implementation of `Map<String, String>`, and yet, if a user tries to mutate an instance, an unchecked exception is thrown. This can legitimately be seen as a safety hazard.

NOTE This is basically the compromise that `Map.of()` has to make: it needs to fully implement the interface and so has to resort to throwing exceptions on mutating method calls.

This is also not the only issue with this approach. Another drawback is that this is, of course, subject to the same basic flaw as we saw for the synchronized case—a mutable object still exists and can be referenced (and mutated) via that route, violating the basic criteria that we were trying to achieve. Let us draw a veil over these attempts and try to look for something better.

5.4 Using ConcurrentHashMap

Having shown a simple map implementation and discussed approaches that we could use to make it concurrent, it's time to meet the `ConcurrentHashMap`. In some ways, this is the easy part: it is an extremely easy-to-use class and in most cases is a drop-in replacement for `HashMap`.

The key point about the `ConcurrentHashMap` is that it is safe for multiple threads to update it at once. To see why we need this, let's see what happens when we have two threads adding entries to a `HashMap` simultaneously (exception handling elided):

```
var map = new HashMap<String, String>();
var SIZE = 10_000;
```

```
Runnable r1 = () -> {
    for (int i = 0; i < SIZE; i = i + 1) {
        map.put("t1" + i, "0");
    }
    System.out.println("Thread 1 done");
};
Runnable r2 = () -> {
    for (int i = 0; i < SIZE; i = i + 1) {
        map.put("t2" + i, "0");
    }
    System.out.println("Thread 2 done");
};
Thread t1 = new Thread(r1);
Thread t2 = new Thread(r2);
t1.start();
t2.start();

t1.join();
t2.join();
System.out.println("Count: "+ map.size());
```

If we run this code, we will see a different manifestation of our old friend, the Lost Update antipattern—the output value for `Count` will be less than `2 * SIZE`. However, in the case of concurrent access to a map, the situation is actually much, much worse.

The most dangerous behavior of `HashMap` under concurrent modification does not always manifest at small sizes. However, if we increase the value of `SIZE` it will, eventually, manifest itself.

If we increase `SIZE` to, say, `1_000_000`, then we are likely to see the behavior. One of the threads making updates to `map` will fail to finish. That's right—one of the threads can (and will) get stuck in an actual infinite loop. This makes `HashMap` totally unsafe for use in multithreaded applications (and the same is true of our example `Dictionary` class).

On the other hand, if we replace `HashMap` with `ConcurrentHashMap`, then we can see that the concurrent version behaves properly—no infinite loops and no instances of Lost Update. It also has the nice property that, no matter what you do to it, map operations will never throw a `ConcurrentModificationException`.

Let's take a very brief look at how this is achieved. It turns out that figure 6.2, which shows the implementation of `Dictionary`, also points the way to a useful multithreaded

generalization of `Map` that is much better than either of our two previous attempts. It is based on the following insight: instead of needing to lock the whole structure when making a change, it's only necessary to lock the hash chain (aka bucket) that's being altered or read.

We can see how this works in figure 6.3. The implementation has moved the lock down onto the individual hash chains. This technique is known as *lock striping*, and it enables multiple threads to access the map, provided they are operating on different chains.

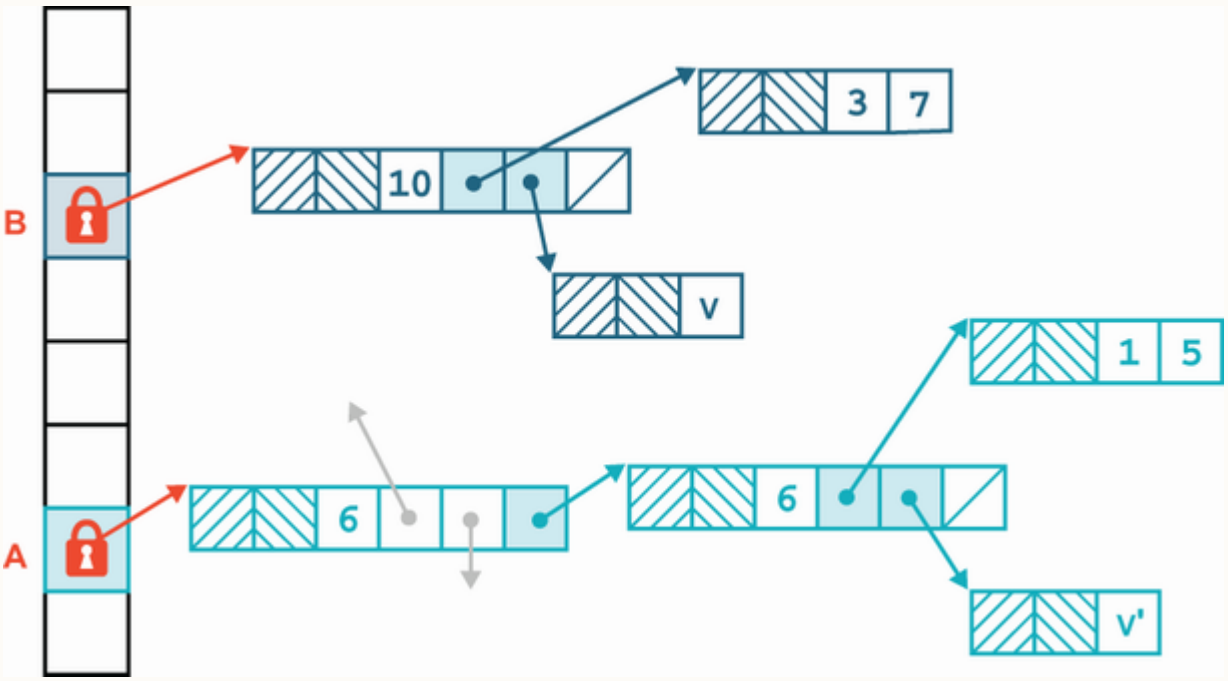


Figure 6.3 Lock striping

Of course, if two threads need to operate on the same chain, then they will still exclude each other, but in general, this provides better throughput than synchronizing the entire map.

NOTE Recall that as the number of elements in the map increases, the table of buckets will resize, meaning that as more and more elements are added to a `ConcurrentHashMap`, it will become able to deal with more and more threads in an efficient manner.

The `ConcurrentHashMap` achieves this behavior, but some additional low-level details exist that most developers won't need to worry about too much. In fact, the implementation of `ConcurrentHashMap` changed substantially in Java 8, and it is now more complex than the design that we have described here.

Using `ConcurrentHashMap` can be almost too simple. In many cases, if you have a multithreaded program and need to share data, then just use a `Map`, and have the implemen-

tation be a `ConcurrentHashMap`. In fact, if there is ever a chance that a `Map` might need to be modified by more than one thread, then you should always use the concurrent implementation. It does use considerably more resources than a plain `HashMap` and will have worse throughput due to the synchronization of some operations. As we'll discuss in chapter 7, however, those inconveniences are nothing when compared to the possibility of a race condition leading to Lost Update or an infinite loop.

Finally, we should also note that `ConcurrentHashMap` actually implements the `ConcurrentMap` interface, which extends `Map`. It originally contained the following new methods to provide thread-safe modifications:

- `putIfAbsent()` —Adds the key-value pair to the `HashMap` if the key isn't already present.
- `remove()` —Safely removes the key-value pair if the key is present.
- `replace()` —The implementation provides two different forms of this method for safe replacement in the `HashMap`.

However, with Java 8, some of these methods were retrofitted to the `Map` interface as default methods, for example:

```
default V putIfAbsent(K key, V value) {
      V v = get(key);
      if (v == null) {
         v = put(key, value);
      }

      return v;
   }
```

The gap between `ConcurrentHashMap` and `Map` has narrowed somewhat in recent versions of Java, but don't forget that despite this, `HashMap` remains thread-unsafe. If you want to share data safely between threads, you should use `ConcurrentHashMap`.

Overall, the `ConcurrentHashMap` is one of the most useful classes in `java.util.concurrent`. It provides additional multithreaded safety and higher performance than synchronization, and it has no serious drawbacks in normal usage. The counterpart to it for `List` is the `CopyOnWriteArrayList`, which we'll discuss next.

6.6 CopyOnWriteArrayList

We can, of course, apply the two unsatisfactory concurrency patterns that we saw in the previous section to `List` as well. Fully synchronized and immutable (but with mutating methods that throw runtime exception) lists are as easy to write down as they are for maps, and they work no better than they do for maps.

Can we do better? Unfortunately, the linear nature of the list is not helpful here. Even in the case of a linked list, multiple threads attempting to modify the list raise the possibility of contention, for example, in workloads that have a large percentage of append operations.

One alternative that does exist is the `CopyOnWriteArrayList` class. As the name suggests, this type is a replacement for the standard `ArrayList` class that has been made thread-safe by the addition of *copy-on-write semantics*. This means that any operations that mutate the list will create a new copy of the array backing the list (as shown in figure 6.4). This also means that any iterators created don't have to worry about any modifications that they didn't expect.

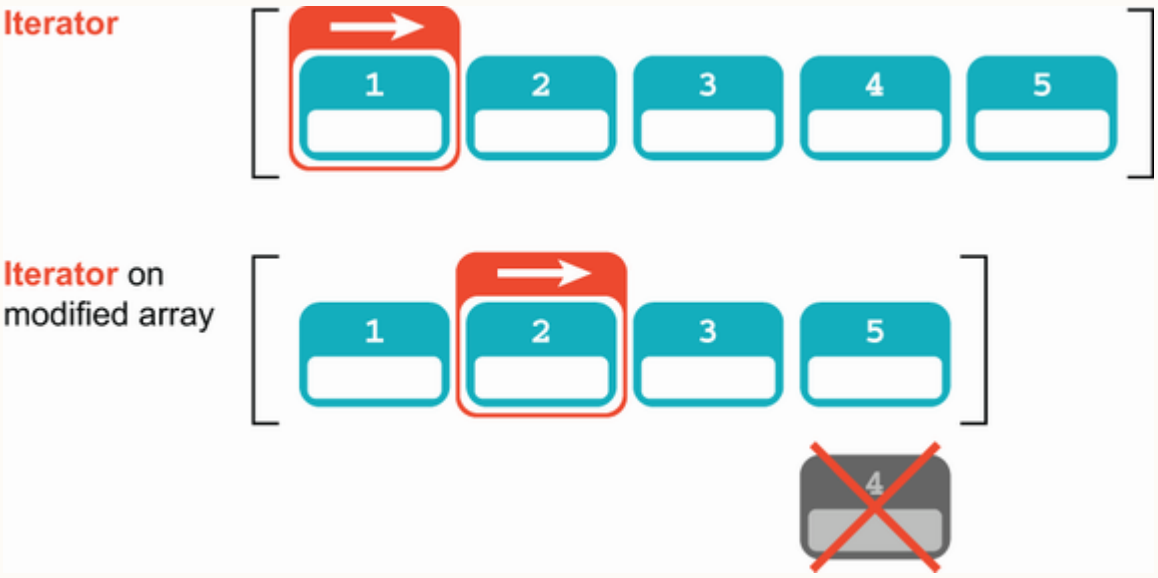


Figure 6.4 Copy-on-write array

The iterators are guaranteed not to throw `ConcurrentModificationException` and will not reflect any additions, removals, or changes to the list since the iterator was created—except, of course, that (as usual in Java) the list elements can still mutate. It is only the list that cannot.

This implementation is usually too expensive for general use but may be a good option when traversal operations vastly outnumber mutations, and when the programmer does

not want the headache of synchronization, yet wants to rule out the possibility of threads interfering with each other.

Let's take a quick look at how the core idea is implemented. The key methods are `iterator()`, which always returns a new `COWIterator` object:

```
public Iterator<E> iterator() {
    return new COWIterator<E>(getArray(), 0);
}
```

and `add()`, `remove()`, and other mutation methods. The mutation methods always replace the delegate array with a new, cloned, and modified copy of the array. Protecting the array must be done within a synchronized block, so the `CopyOnWriteArrayList` class has an internal lock that is just used as a monitor (and note the comment on it), as shown here:

```
/**
 * The lock protecting all mutators.  (We have a mild preference
 * for builtin monitors over ReentrantLock when either will do.)
 */
final transient Object lock = new Object();

private transient volatile Object[] array;
```

Then, operations such as `add()` can be protected as follows:

```
public boolean add(E e) {
    synchronized (lock) {
        Object[] es = getArray();
        int len = es.length;
        es = Arrays.copyOf(es, len + 1);
        es[len] = e;
        setArray(es);
        return true;
    }
}
```

This makes the `CopyOnWriteArrayList` less efficient than the `ArrayList` for general operations, for several reasons:

- Synchronization of mutation operations.
- Volatile storage (i.e., `array`).
- `ArrayList` will only allocate memory when a resize of the underlying array is required; `CopyOnWriteArrayList` allocates and copies on every mutation.

Creating the iterator stores a reference to the array as it exists at that point in time. Further modifications to the list cause a new copy to be created, so the iterator will then be pointing at a past version of the array, as shown next:

```
    static final class COWIterator<E> implements ListIterator<E> {
      /** Snapshot of the array */
      private final Object[] snapshot;
      /** Index of element to be returned by subsequent call to next */
      private int cursor;

      COWIterator(Object[] es, int initialCursor) {
         cursor = initialCursor;
         snapshot = es;
      }
      // ...
    }
```

Note that `COWIterator` implements `ListIterator` and so, according to the interface contract, is required to support list mutation methods, but for simplicity's sake, the mutators all throw `UnsupportedOperationException`.

The approach taken by `CopyOnWriteArrayList` to shared data may be useful when a quick, consistent snapshot of data (which may occasionally be different between readers) is more important than perfect synchronization. This is seen reasonably often in scenarios that involve non-mission-critical data, and the copy-on-write approach avoids the performance hit associated with synchronization.

Let's look at an example of copy-on-write in action in the next listing.

Listing 6.3 Copy-on-write example

```
      var ls = new CopyOnWriteArrayList(List.of(1, 2, 3));
      var it = ls.iterator();
      ls.add(4);
      var modifiedIt = ls.iterator();
      while (it.hasNext()) {
         System.out.println("Original: "+ it.next());
```

```
        }
        while (modifiedIt.hasNext()) {
            System.out.println("Modified: "+ modifiedIt.next());
        }
```

This code is specifically designed to illustrate the behavior of an `Iterator` under copy-on-write semantics. It produces output like this:

```
Original: 1
Original: 2
Original: 3
Modified: 1
Modified: 2
Modified: 3
Modified: 4
```

In general, the use of the `CopyOnWriteArrayList` class does require a bit more thought than using `ConcurrentHashMap`, which is basically a drop-in concurrent replacement for `HashMap` because of performance issues—the copy-on-write property means that if the list is altered, the entire array must be copied. If changes to the list are common, compared to read accesses, this approach won't necessarily yield high performance.

In general, the `CopyOnWriteArrayList` makes different trade-offs than `synchronizedList()`. The latter synchronizes on all operations, so reads from different threads can block each other, which is not true for a COW data structure. On the other hand, `CopyOnWriteArrayList` copies the backing array on every mutation, whereas the synchronized version does so only when the backing array is full (the same behavior as `ArrayList`). However, as we'll say repeatedly in chapter 7, reasoning about code from first principles is extremely difficult—the only way to reliably get well-performing code is to test, retest, and measure the results.

Later, in chapter 15, we'll meet the concept of a *persistent data structure*, which is another way of approaching concurrent data handling. The Clojure programming language makes very heavy use of persistent data structures, and the `CopyOnWriteArrayList` (and `CopyOnWriteArraySet`) is one example implementation of them.

Let's move on. The next major common building block of concurrent code in `java.util.concurrent` is the `Queue`. This is used to hand off work elements between threads, and it is used as the basis for many flexible and reliable multithreaded designs.

6.7 Blocking queues

The queue is a wonderful abstraction for concurrent programming. The queue provides a simple and reliable way to distribute processing resources to work units (or to assign work units to processing resources, depending on how you want to look at it).

A number of patterns in multithreaded Java programming rely heavily on the thread-safe implementations of `Queue`, so it's important that you fully understand it. The basic `Queue` interface is in `java.util`, because it can be an important pattern, even in single-threaded programming, but we'll focus on the multithreaded use cases.

One very common use case, and the one we'll focus on, is the use of a queue to transfer work units between threads. This pattern is often ideally suited for the simplest concurrent extension of `Queue`—the `BlockingQueue`.

The `BlockingQueue` is a queue that has the following two additional special properties:

- When trying to `put()` to the queue, it will cause the putting thread to wait for space to become available if the queue is full.
- When trying to `take()` from the queue, it will cause the taking thread to block if the queue is empty.

These two properties are very useful because if one thread (or pool of threads) is outstripping the ability of the other to keep up, the faster thread is forced to wait, thus regulating the overall system. This is illustrated in figure 6.5.

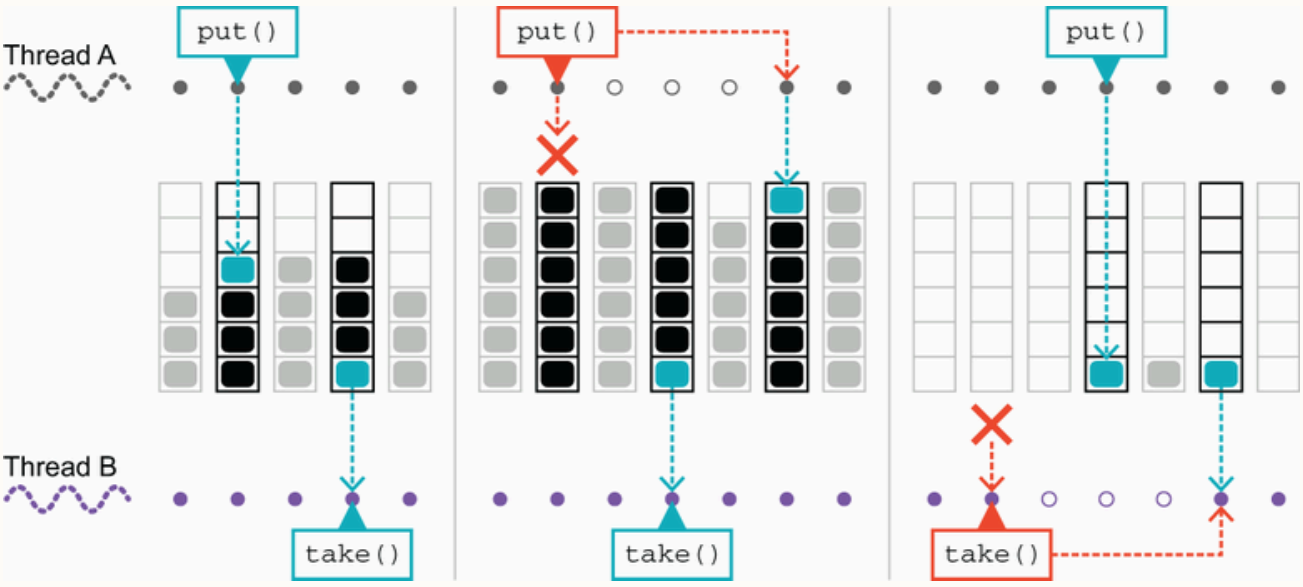


Figure 6.5 The `BlockingQueue`

Java ships with two basic implementations of the `BlockingQueue` interface: the `LinkedBlockingQueue` and the `ArrayBlockingQueue`. They offer slightly different properties; for example, the array implementation is very efficient when an exact bound is known for the size of the queue, whereas the linked implementation may be slightly faster under some circumstances.

However, the real difference between the implementations is in the implied semantics. Although the linked variant can be constructed with a size limit, it is usually created without one, which leads to an object with a queue size of `Integer.MAX_VALUE`. This is effectively infinite—a real application would never be able to recover from a backlog of over two billion items in one of its queues.

So, although in theory the `put()` method on `LinkedBlockingQueue` can block, in practice, it never does. This means that the threads that are writing to the queue can effectively proceed at an unlimited rate.

In contrast, the `ArrayBlockingQueue` has a fixed size for the queue—the size of the array that backs it. If the producer threads are putting objects into the queue faster than they are being processed by receivers, at some point the queue will fill completely, further attempts to call `put()` will block, and the producer threads will be forced to slow their rate of task production.

This property of the `ArrayBlockingQueue` is one form of what is known as *back pressure*, which is an important aspect of engineering concurrent and distributed systems.

Let's see the `BlockingQueue` in action in an example: altering the account example to use queues and threads. The aim of the example will be to get rid of the need to lock both account objects. The basic architecture of the application is shown in figure 6.6.

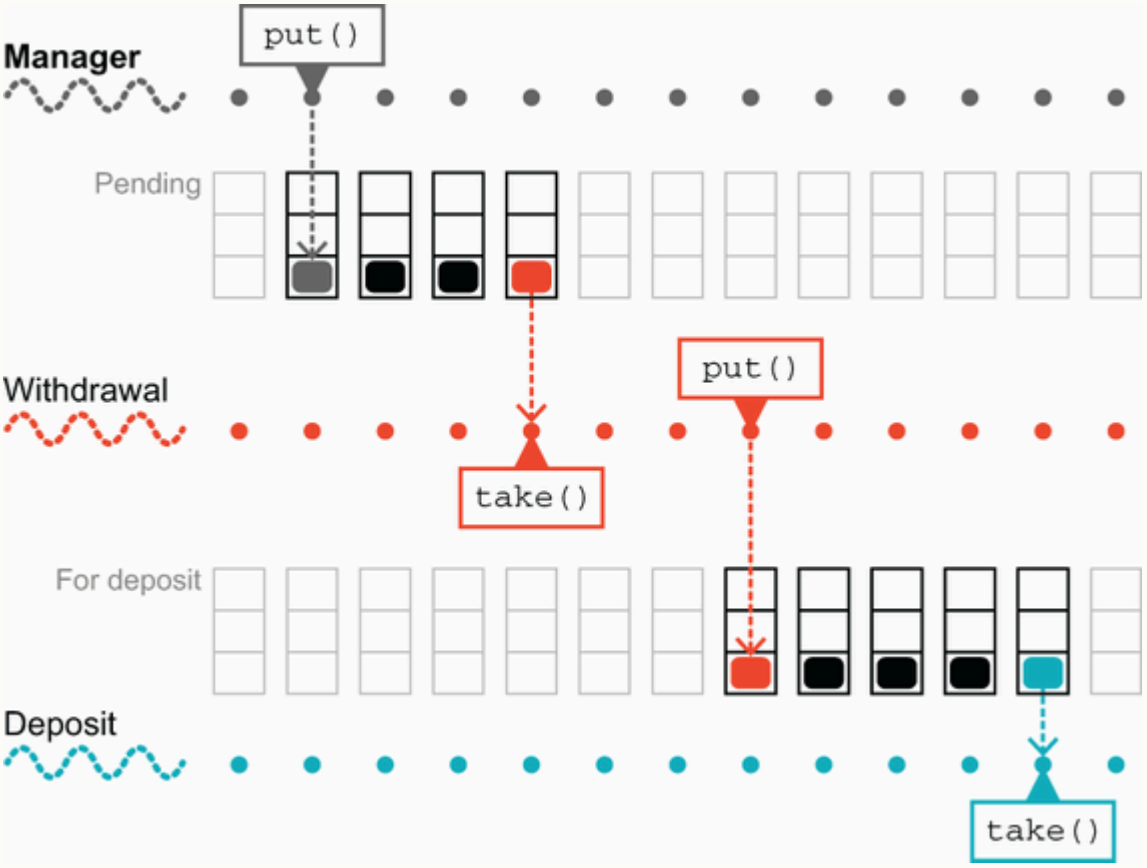


Figure 6.6 Handling accounts with queues

We start by introducing a `AccountManager` class with these fields, as shown in the next listing.

Listing 6.4 The `AccountManager` class

```
public class AccountManager {
   private ConcurrentHashMap<Integer, Account> accounts =
      new ConcurrentHashMap<>();
   private volatile boolean shutdown = false;

   private BlockingQueue<TransferTask> pending =
      new LinkedBlockingQueue<>();
   private BlockingQueue<TransferTask> forDeposit =
      new LinkedBlockingQueue<>();
   private BlockingQueue<TransferTask> failed =
      new LinkedBlockingQueue<>();

   private Thread withdrawals;
   private Thread deposits;
```

The blocking queues contain `TransferTask` objects, which are simple data carriers that denote the transfer to be made, as shown next:

```
public class TransferTask {
    private final Account sender;
    private final Account receiver;
    private final int amount;

    public TransferTask(Account sender, Account receiver, int amount) {
        this.sender = sender;
        this.receiver = receiver;
        this.amount = amount;
    }

    public Account sender() {
        return sender;
    }

    public int amount() {
        return amount;
    }

    public Account receiver() {
        return receiver;
    }

    // Other methods elided
}
```

There is no additional semantics for the transfer—the class is just a dumb *data carrier* type.

NOTE The `TransferTask` type is very simple and, in Java 17, could be written as a record type (which we met in chapter 3)

The `AccountManager` class provides functionality for accounts to be created and for transfer tasks to be submitted, as illustrated here:

```
    public Account createAccount(int balance) {
        var out = new Account(balance);
        accounts.put(out.getAccountId(), out);
        return out;
    }

    public void submit(TransferTask transfer) {
        if (shutdown) {
```

```
            return false;
        }
        return pending.add(transfer);
    }
```

The real work of the `AccountManager` is handled by the two threads that manage the transfer tasks between the queues. Let's look at the withdraw operation first:

```
    public void init() {
        Runnable withdraw = () -> {
            boolean interrupted = false;
            while (!interrupted || !pending.isEmpty()) {
                try {
                    var task = pending.take();
                    var sender = task.sender();
                    if (sender.withdraw(task.amount())) {
                        forDeposit.add(task);
                    } else {
                        failed.add(task);
                    }
                } catch (InterruptedException e) {
                    interrupted = true;
                }
            }
            deposits.interrupt();
        };
```

The deposit operation is defined similarly, and then we initialize the account manager with the tasks:

```
        Runnable deposit = () -> {
            boolean interrupted = false;
            while (!interrupted || !forDeposit.isEmpty()) {
                try {
                    var task = forDeposit.take();
                    var receiver = task.receiver();
                    receiver.deposit(task.amount());
                } catch (InterruptedException e) {
                    interrupted = true;
                }
            }
        };
```

```
    init(withdraw, deposit);
  }
```

The package-private overload of the `init()` method is used to start the background threads. It exists as a separate method to allow for easier testing, as follows:

```
  void init(Runnable withdraw, Runnable deposit) {
    withdrawals = new Thread(withdraw);
    deposits = new Thread(deposit);
    withdrawals.start();
    deposits.start();
  }
```

We need some code to drive this:

```
    var manager = new AccountManager();
    manager.init();
    var acc1 = manager.createAccount(1000);
    var acc2 = manager.createAccount(20_000);

    var transfer = new TransferTask(acc1, acc2, 100);
    manager.submit(transfer);                    (1)
    Thread.sleep(5000);                          (2)
    System.out.println(acc1);
    System.out.println(acc2);
    manager.shutdown();
    manager.await();
```

(1) Submits the transfer from acc1 to acc2

(2) Sleeps to allow time for the transfer to execute

This produces output like this:

```
Account{accountId=1, balance=900.0,
  lock=java.util.concurrent.locks.ReentrantLock@58372a00[Unlocked]}
Account{accountId=2, balance=20100.0,
  lock=java.util.concurrent.locks.ReentrantLock@4dd8dc3[Unlocked]}
```

However, the code as written does not execute cleanly, despite the calls to `shutdown()` and `await()` because of the blocking nature of the calls used. Let's look at figure 6.7 to see why.

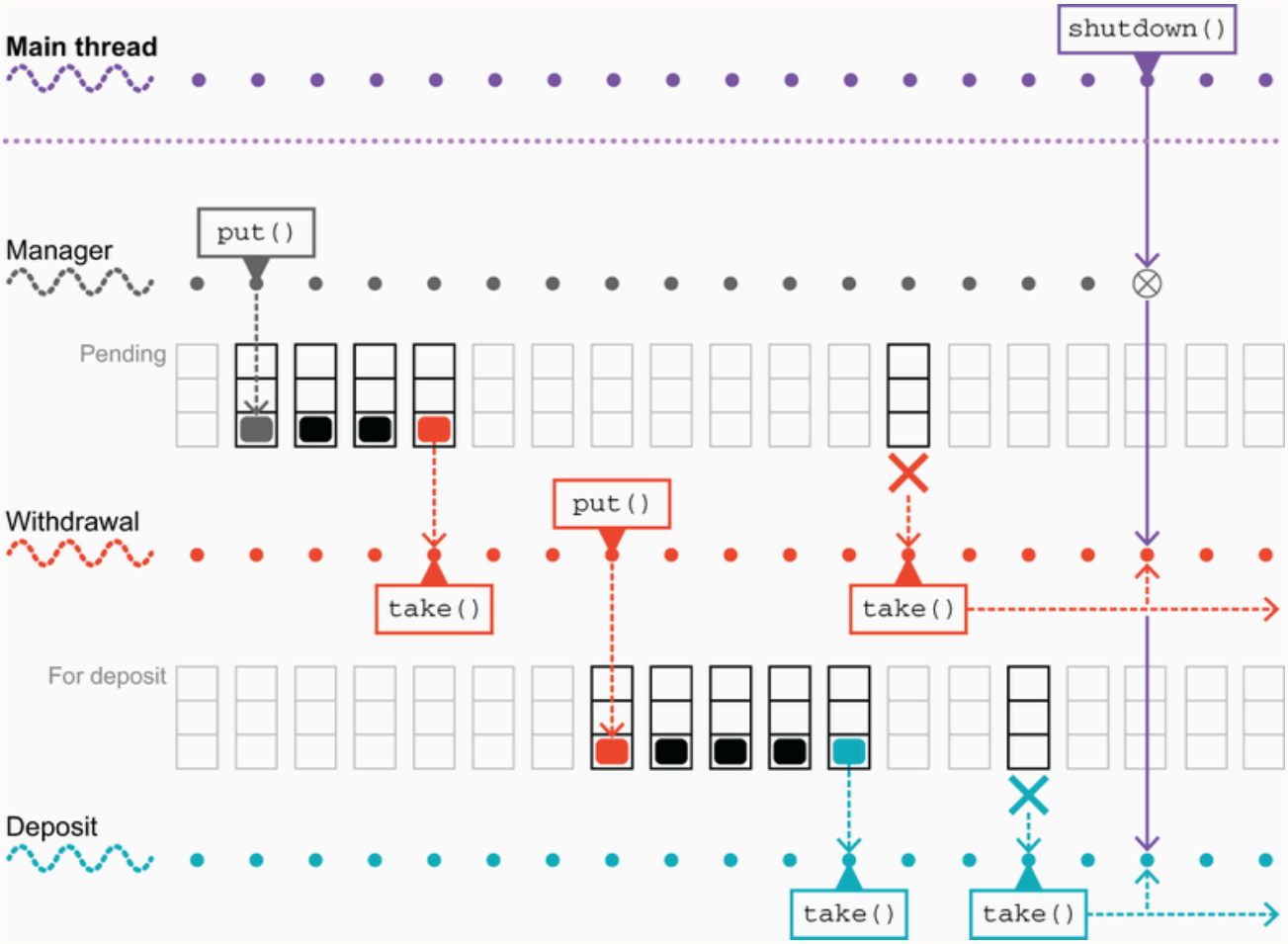


Figure 6.7 An incorrect shutdown sequence

When the main code calls `shutdown()`, the volatile boolean flag is flipped to true, so every subsequent read of the boolean will see the value as true. Unfortunately, both the withdrawal and depositing threads are blocked in calls to `take()` because the queues are empty. If an object was somehow placed into the `pending` queue, then the withdrawal thread would process it and then place the object into the `forDeposit` queue (assuming the withdrawal succeeds). The withdrawal thread would at this point exit the `while` loop, and the thread would terminate normally.

In turn, the depositing thread will now see the object in the `forDeposit` queue and will wake up, take it, process it, and then exit its own `while` loop and also terminate normally. However, this clean termination process depends on there still being tasks in the queue. In the edge case of an empty queue, the threads will sit in their blocking `take()` calls forever. To solve this issue, let's explore the full range of methods that are provided by the blocking queue implementations.

7.1 Using BlockingQueue APIs

The interface `BlockingQueue` actually provides three separate strategies for interacting with it. To understand the differences between the strategies, consider the possible behaviors that an API could display in the following scenario: a thread attempts to insert an item in a capacity-restricted queue that is currently unable to accommodate the item (i.e., the queue is full).

Logically, we have the following three possibilities. The insertion call could

- Block until space in the queue frees up
- Return a value (perhaps Boolean false) indicating failure
- Throw an exception

The same three possibilities would, of course, occur in the converse situation (attempting to take an item from an empty queue). The first of these possibilities is realized by the `take()` and `put()` methods that we have already met.

NOTE The second and third are the options provided by the `Queue` interface, which is the super interface of `BlockingQueue`.

The second option provides a nonblocking API that returns special values and is manifested in the methods `offer()` and `poll()`. If insertion into the queue cannot be completed, then `offer()` fails fast and returns false. The programmer must examine the return code and take appropriate action.

Similarly, `poll()` immediately returns `null` on failure to retrieve from the queue. It might seem a bit odd to have a nonblocking API on a class explicitly named `BlockingQueue`, but it is actually useful (and also required as a consequence of the inheritance relationship between `BlockingQueue` and `Queue`).

In fact, `BlockingQueue` provides an additional overload of the nonblocking methods. These methods provide the capability of polling or offering with a timeout, to allow the thread encountering issues to back out from its interaction with the queue and do something else instead.

We can modify the `AccountManager` in listing 6.4 to use the nonblocking APIs with a timeout, like so:

```
Runnable withdraw = () -> {
  LOOP:
```

```
    while (!shutdown) {
      try {
        var task = pending.poll(5,              (1)
                                TimeUnit.SECONDS);
        if (task == null) {
          continue LOOP;                        (2)
        }
        var sender = task.sender();
        if (sender.withdraw(task.amount())) {
          forDeposit.put(task);
        } else {
          failed.put(task);
        }
      } catch (InterruptedException e) {
        // Log at critical and proceed to next item
      }
    }
    // Drain pending queue to failed or log
  };
```

(1) If the timer expires, poll() returns null.

(2) Explicit use of a Java loop label to make it clear what is being continued.

Similar modifications should be made for the deposit thread as well.

This solves the shutdown problem that we outlined in the previous subsection, because now the threads cannot block forever in the retrieval methods. Instead, if no object arrives before the timeout, then the poll will still return and provide the value `null`. The test then continues the loop, but the visibility guarantees of the volatile Boolean ensure that the `while` loop condition is now met and the loop is exited and the thread shuts down cleanly. This means that overall, once the `shutdown()` method has been called, the `AccountManager` will shut down in bounded time, which is the behavior we want.

To conclude the discussion of the APIs of `BlockingQueue`, we should look at the third approach we mentioned earlier: methods that throw exceptions if the queue operation cannot immediately complete. These methods, `add()` and `remove()`, are, frankly, problematic for several reasons, not least of which is that the exceptions they throw on failure (`IllegalStateException` and `NoSuchElementException` respectively) are runtime exceptions and so do not need to be explicitly handled.

The problems with the exception-throwing API are deeper than just this, though. A general principle in Java states that exceptions are to be used to deal with exceptional circumstances, that is, those that a program does not normally consider to be part of normal operation. The situation of an empty queue is, however, an entirely possible circumstance. So throwing an exception in response to it is a violation of the principle that is sometimes expressed as "Don't use exceptions for flow control."

Exceptions are, in general, quite expensive to use, due to stack trace construction when the exception is instantiated and stack unwinding during the throw. It is good practice not to create an exception unless it is going to immediately be thrown. For these reasons, we do recommend against using the exception-throwing form of the `BlockingQueue` APIs.

7.2 Using WorkUnit

The `Queue` interfaces are all generic: they're `Queue<E>`, `BlockingQueue<E>`, and so on. Although it may seem strange, it's sometimes wise to exploit this and introduce an artificial container class to wrap the items of work.

For example, if you have a class called `MyAwesomeClass` that represents the units of work that you want to process in a multithreaded application, then rather than having this:

```
BlockingQueue<MyAwesomeClass>
```

it can be better to have this:

```
BlockingQueue<WorkUnit<MyAwesomeClass>>
```

where `WorkUnit` (or `QueueObject`, or whatever you want to call the container class) is a packaging class that may look something like this:

```
public class WorkUnit<T> {
   private final T workUnit;

   public T getWork() {
      return workUnit;
   }

   public WorkUnit(T workUnit) {
```

```
        this.workUnit = workUnit;
    }

    // ... other methods elided
}
```

The reason for doing this is that this level of indirection provides a place to add additional metadata without compromising the conceptual integrity of the contained type (`MyAwesomeClass`, in this example). In figure 6.8, we can see how the external metadata wrapper works.



Figure 6.8 Use of a work unit as a metadata wrapper

This is surprisingly useful. Use cases where additional metadata is helpful are abundant. Here are a few examples:

- Testing (such as showing the change history for an object)
- Performance indicators (such as time of arrival or quality of service)
- Runtime system information (such as how this instance of `MyAwesomeClass` has been routed)

It can be much harder to add in this indirection after the fact. If you later discover that more metadata is needed in certain circumstances, it can be a major refactoring job to add in what would have been a simple change in the `WorkUnit` class. Let's move on to discuss futures, which are a way of representing a placeholder for an in-progress (usually on another thread) task in Java.

6.8 Futures

The interface `Future` in `java.util.concurrent` is a simple representation of an asynchronous task: it is a type that holds the result from a task that may not have finished yet but may at some point in the future. The primary methods on a `Future` follow:

- `get()` —Gets the result. If the result isn't yet available, will block until it is.

- `isDone()` —Allows the caller to determine whether the computation has finished. It is nonblocking.
- `cancel()` —Allows the computation to be canceled before completion.

There's also a version of `get()` that takes a timeout, which won't block forever, in a similar manner to the `BlockingQueue` methods with timeouts that we met earlier. The next listing shows a sample use of a `Future` in a prime number finder.

Listing 6.5 Finding prime numbers using a `Future`

```
Future<Long> fut = getNthPrime(1_000_000_000);
try {
   long result = fut.get(1, TimeUnit.MINUTES);
   System.out.println("Found it: " + result);
} catch (TimeoutException tox) {
   // Timed out - better cancel the task
   System.err.println("Task timed out, cancelling");
   fut.cancel(true);
} catch (InterruptedException e) {
   fut.cancel(true);
   throw e;
} catch (ExecutionException e) {
   fut.cancel(true);
   e.getCause().printStackTrace();
}
```

In this snippet, you should imagine that `getNthPrime()` returns a `Future` that is executing on some background thread (or even on multiple threads)—perhaps on one of the executor frameworks we'll discuss later in the chapter.

The thread running the snippet enters a get-with-timeout and blocks for up to 60 seconds for a response. If no response is received, then the thread loops and enters another blocking wait. Even on modern hardware, this calculation may be running for a long time, so you may need to use the `cancel()` method after all (although the code as written does not provide any mechanism to cancel our request).

As a second example, let's consider nonblocking I/O. Figure 6.9 shows the `Future` in action to allow us to use a background thread for I/O.

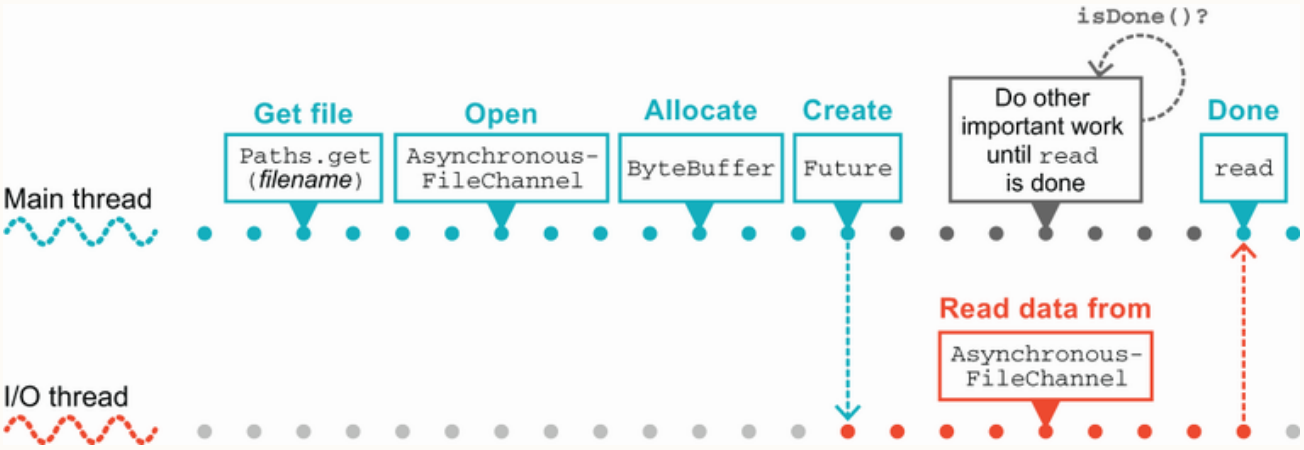


Figure 6.9 Using `Future` in Java

This API has been around for a while—it was introduced in Java 7—and it allows the user to do nonblocking concurrency like this:

```
try {
   Path file = Paths.get("/Users/karianna/foobar.txt");

   var channel = AsynchronousFileChannel.open(file);       ❶

   var buffer = ByteBuffer.allocate(1_000_000);            ❷
   Future<Integer> result = channel.read(buffer, 0);       ❷

   BusinessProcess.doSomethingElse();                      ❸

   var bytesRead = result.get();                           ❹
   System.out.println("Bytes read [" + bytesRead + "]");
} catch (IOException | ExecutionException | InterruptedException e) {
   e.printStackTrace();
}
```

❶ Opens the file asynchronously

❷ Requests a read of up to one million bytes

❸ Does something else

❹ Gets the result when ready

This structure allows the main thread to `doSomethingElse()` while the I/O operation is proceeding on another thread—one that is managed by the Java runtime. This is a useful

approach, but it requires support in the library that provides the capability. This can be somewhat limited—and what if we want to create our own asynchronous workflows?

18.1 CompletableFuture

The Java `Future` type is defined as an interface, rather than a concrete class. Any API that wants to use a `Future`-based style has to supply a concrete implementation of `Future`.

These can be challenging for some developers to write and represents an obvious gap in the toolkit, so from Java 8 onward, a new approach to futures was included in the JDK—a concrete implementation of `Future` that enhances capabilities and in some ways is more similar to futures in other languages (e.g., Kotlin and Scala).

The class is called `CompletableFuture`—it is a concrete type that implements the `Future` interface and provides additional functionality and is intended as a simple building block for building asynchronous applications. The central idea is that we can create instances of the `CompletableFuture<T>` type (it is generic in the type of the value that will be returned), and the object that is created represents the `Future` in an *uncompleted* (or "unfulfilled") state.

Later, any thread that has a reference to the completable `Future` can call `complete()` on it and provide a value—this completes (or "fulfills") the future. The completed value is immediately visible to all threads that are blocked on a `get()` call. After completion, any further calls to `complete()` are ignored.

The `CompletableFuture` cannot cause different threads to see different values. The `Future` is either uncompleted or completed, and if it is completed, the value it holds is the value provided by the first thread to call `complete()`.

This is obviously not immutability—the state of the `CompletableFuture` does change over time. However, it changes only *once*—from uncompleted to completed. There is no possibility of an inconsistent state being seen by different threads.

NOTE Java's `CompletableFuture` is similar to a *promise*, as seen in other languages (such as JavaScript), which is why we call out the alternative terminology of "fulfilling a promise" as well as "completing a future."

Let's look at an example and implement `getNthPrime()`, which we met earlier:

```
public static Future<Long> getNthPrime(int n) {
   var numF = new CompletableFuture<Long>();     ❶

   new Thread( () -> {                           ❷
      long num = NumberService.findPrime(n);     ❸
      numF.complete(num);
   } ).start();

   return numF;
}
```

❶ Creates the completable Future in an uncompleted state

❷ Creates and starts a new thread that will complete the Future

❸ The actual calculation of the prime number

The method `getNthPrime()` creates an "empty" `CompletableFuture` and returns this container object to its caller. To drive this, we do need some code to call `getNthPrime()` —for example, the code shown in listing 6.5.

One way to think about `CompletableFuture` is by analogy with client/server systems. The `Future` interface provides only a query method— `isDone()` and a blocking `get()`. This is playing the role of the client. An instance of `CompletableFuture` plays the role of the server side—it provides full control over the execution and completion of the code that is fulfilling the future and providing the value.

In the example, `getNthPrime()` evaluates the call to the number service in a separate thread. When this call returns, we complete the future explicitly.

A slightly more concise way to achieve the same effect is to use the `CompletableFuture.supplyAsync()` method, passing a `Callable<T>` object representing the task to be executed. This call makes use of an application wide thread pool that is managed by the concurrency library, as follows:

```
public static Future<Long> getNthPrime(int n) {
   return CompletableFuture.supplyAsync(
      () -> NumberService.findPrime(n));
}
```

This concludes our initial tour of the concurrent data structures that are some of the main building blocks that provide the raw materials for developing solid multithreaded applications.

NOTE We will have more to say about the `CompletableFuture` later in the book, specifically in the chapters that discuss advanced concurrency and the interplay with functional programming.

Next, we'll introduce the *executors* and threadpools that provide a higher-level and more convenient way to handle execution than the raw API based on `Thread`.

6.9 Tasks and execution

The class `java.lang.Thread` has existed since Java 1.0—one of the original talking points of the Java language was built-in, language-level support for multithreading. It is powerful and expresses concurrency in a form that is close to the underlying operating system support. However, it is a fundamentally low-level API for handling concurrency.

This low-level nature makes it hard for many programmers to work with correctly or efficiently. Other languages that were released after Java learned from Java's experience with threads and built upon them to provide alternative approaches. Some of those approaches have, in turn, influenced the design of `java.util.concurrent` and later innovations in Java concurrency.

In this case, our immediate goal is to have tasks (or work units) that can be executed without spinning up a new thread for each one. Ultimately, this means that the tasks have to be modeled as code that can be called rather than directly represented as a thread.

Then, these tasks can be scheduled on a shared resource—a pool of threads—that executes a task to completion and then moves on to the next task. Let's take a look at how we model these tasks.

6.9.1 Modeling tasks

In this section, we'll look at two different ways of modeling tasks: the `Callable` interface and the `FutureTask` class. We could also consider `Runnable`, but it is not always that useful, because the `run()` method does not return a value, and, therefore, it can perform work only via side effects.

One other aspect of the task modeling is important but may not be obvious— the notion that if we assume that our thread capacity is finite, tasks must definitely complete in bounded time.

If we have the possibility of an infinite loop, some tasks could "steal" an executor thread from the pool, and this would reduce the overall capacity for all tasks from then on. Over time, this could eventually lead to exhaustion of the thread pool resource and no further work being possible. As a result, we must be careful that any tasks we construct do actually obey the "terminate in finite time" principle.

Callable interface

The `Callable` interface represents a very common abstraction. It represents a piece of code that can be called and returns a result. Despite being a straightforward idea, this is actually a subtle and powerful concept that can lead to some extremely useful patterns.

One typical use of a `Callable` is the lambda expression (or an anonymous implementation). The last line of this snippet sets `s` to be the value of `out.toString()`:

```
var out = getSampleObject();
Callable<String> cb = () -> out.toString();

String s = cb.call();
```

Think of a `Callable` as being a deferred invocation of the single method, `call()`, which the lambda provides.

FutureTask class

The `FutureTask` class is one commonly used implementation of the `Future` interface, which also implements `Runnable`. As we'll see, this means that a `FutureTask` can be fed to executors. The API of `FutureTask` is basically that of `Future` and `Runnable` combined: `get()`, `cancel()`, `isDone()`, `isCancelled()`, and `run()`, although the last of these —the one that does the actual work—would be called by the executor, rather than directly by client code.

Two convenience constructors for `FutureTask` are provided: one that takes a `Callable` and one that takes a `Runnable` (which uses `Executors.callable()` to convert the `Runnable` to a `Callable`). This suggests a flexible approach to tasks, allowing a job to be written as a `Callable` then wrapped into a `FutureTask` that can then be scheduled

(and cancelled, if necessary) on an executor, due to the `Runnable` nature of `FutureTask`.

The class provides a simple state model for tasks and management of a task through that model. The possible state transitions are shown in figure 6.10

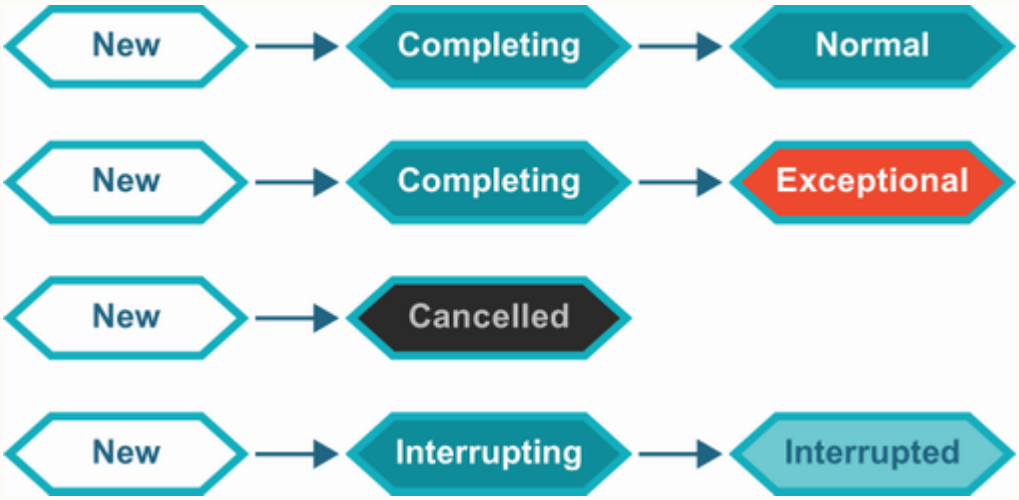


Figure 6.10 State model for tasks

This is sufficient for a wide range of ordinary execution possibilities. Let's meet the standard executors that are provided by the JDK.

9.2 Executors

A couple of standard interfaces are used to describe the threadpools present in the JDK. The first is `Executor`, which is very simple and defined like this:

```
public interface Executor {

  /**
   * Executes the given command at some time in the future. The command
   * may execute in a new thread, in a pooled thread, or in the calling
   * thread, at the discretion of the {@code Executor} implementation.
   *
   * @param command the runnable task
   * @throws RejectedExecutionException if this task cannot be
   * accepted for execution
   * @throws NullPointerException if command is null
   */
  void execute(Runnable command);
}
```

You should note that although this interface has only a single abstract method (i.e., it is a so-called SAM type), it is *not* tagged with the annotation `@FunctionalInterface`. It can still be used as the target type for a lambda expression, but it is not intended for use in functional programming.

In fact, the `Executor` is not widely used—far more common is the `ExecutorService` interface that extends `Executor` and adds `submit()` as well as several *life cycle* methods, such as `shutdown()`.

To help the developer instantiate and work with some standard threadpools, the JDK provides the `Executors` class, which is a collection of static helper methods (mostly factories). Four of the most commonly used methods follow:

```
newSingleThreadExecutor()
newFixedThreadPool(int nThreads)
newCachedThreadPool()
newScheduledThreadPool(int corePoolSize)
```

Let's look at each of these in turn. Later in the book, we will dive into some of the other, more complex, possibilities that are also provided.

9.3 Single-threaded executor

The simplest of the executors is the single-threaded executor. This is essentially an encapsulated combination of a single thread and a task queue (which is a blocking queue).

Client code places an executable task onto the queue via `submit()`. The single execution thread then takes the tasks one at a time and runs each to completion before taking the next task.

NOTE The executors are not implemented as distinct types but instead represent different parameter choices when constructing an underlying threadpool.

Any tasks that are submitted while the execution thread is busy are queued until the thread is available. Because this executor is backed by a single thread, if the previously mentioned "terminate in finite time" condition is violated, it means that no subsequently submitted job will ever run.

NOTE This version of the executor is often useful for testing because it can be made more deterministic that other forms.

Here is a very simple example of how to use the single-threaded executor:

```
var pool = Executors.newSingleThreadExecutor();
Runnable hello = () -> System.out.println("Hello world");
pool.submit(hello);
```

The `submit()` call hands off the runnable task by placing it on the executor's job queue. That job submission is nonblocking (unless the job queue is full).

However, care must still be taken—for example, if the main thread exits immediately, the submitted job may not have had time to be collected by the pool thread and may not run. Instead of exiting straightaway, it is wise to call the `shutdown()` method on the executor first.

The details can be found in the `ThreadPoolExecutor` class, but basically this method starts an *orderly shutdown* in which previously submitted tasks are executed but no new tasks will be accepted. This effectively solves the issues we saw in listing 6.4 about draining the queues of pending transactions.

NOTE The combination of a task that loops infinitely and an orderly shutdown request will interact badly, resulting in a threadpool that never shuts down.

Of course, if the single-threaded executor was all that was needed, there wouldn't be a need to develop a deep understanding of concurrent programming and its challenges. So, we should also look at the alternatives that utilize multiple-executor threads.

9.4 Fixed-thread pool

The fixed-thread pool, obtained via one of the variants of `Executors.newFixedThreadPool()`, is essentially the multiple-thread generalization of the single-threaded executor. At creation time, the user supplies an explicit thread count, and the pool is created with that many threads.

These threads will be reused to run multiple tasks, one after another. The design prevents users having to pay the cost of thread creation. As with the single-threaded variant, if all threads are in use, new tasks are stored in a blocking queue until a thread becomes free.

This version of the threadpool is particularly useful if task flow is stable and known and if all submitted jobs are roughly the same size, in terms of computation duration. It is,

once again, most easily created from the appropriate factory method, as shown here:

```
var pool = Executors.newFixedThreadPool(2);
```

This will create an explicit thread pool backed by two executor threads. The two threads will take turns accepting tasks from the queue in a nondeterministic manner. Even if there is a strict temporal (time-based) ordering of when tasks are submitted, there is no guarantee of which thread will handle a given task.

One consequence of this is that in a situation like that shown in figure 6.11, the tasks in the downstream queue cannot be relied upon to be accurately temporally ordered, even if the tasks in the upstream queue are.

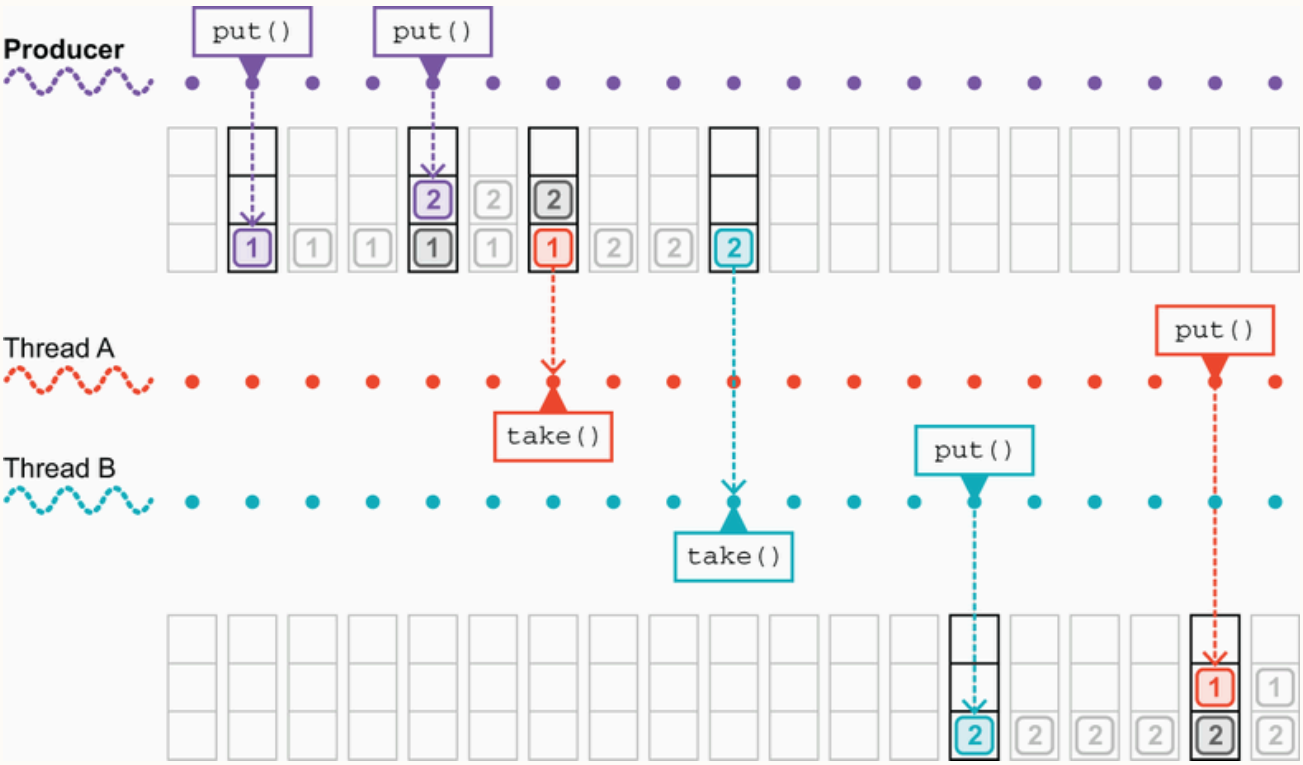


Figure 6.11 A threadpool and two queues

The fixed threadpool has its uses, but it is not the only game in town. For one thing, if the executor threads in it die, they are not replaced. If the possibility exists of the submitted jobs throwing a runtime exception, this can lead to the threadpool starving. Let's look at another alternative, which makes different trade-offs but which can avoid this possibility.

9.5 Cached thread pool

The fixed threadpool is often used when the activity pattern of the workload is known and fairly stable. However, if the incoming work is more uneven or bursty, then a pool

with a fixed number of threads is likely to be suboptimal.

The `CachedThreadPool` is an unbounded pool, that will reuse threads if they are available but otherwise will create new threads as required to handle incoming tasks, as shown here:

```
var pool = Executors.newCachedThreadPool();
```

Threads are kept in the idle cache for 60 seconds, and if they are still present at the end of that period, they will be removed from the cache and destroyed.

It is, of course, still very important that the tasks do actually terminate. If not, then the threadpool will, over time, create more and more threads and consume more and more of the machine's resources and eventually crash or become unresponsive.

In general, the trade-off between fixed-size thread pools and cached thread pools is largely about reusing threads versus creating and destroying threads to achieve different effects. The design of the `CachedThreadPool` should give better performance with small asynchronous tasks as compared to the performance achieved from fixed-size pools. However, as always, if the effect is thought to be significant, proper performance testing much be undertaken.

9.6 ScheduledThreadPoolExecutor

The final example of an executor that we'll look at is a little bit different. This is the `ScheduledThreadPoolExecutor`, sometimes referred to as an STPE, as shown here:

```
ScheduledExecutorService pool = Executors.newScheduledThreadPool(4);
```

Note that the return type, which we've explicitly called out here, is `ScheduledExecutorService`. This is different from the other factory methods, which return `ExecutorService`.

NOTE The `ScheduledThreadPoolExecutor` is a surprisingly capable choice of executor and can be used across a wide range of circumstances.

The scheduled service extends the usual executor service and adds a small amount of new capabilities: `schedule()`, which runs a one-off task after a specified delay, and two

methods for scheduling periodic (i.e., repeating tasks)— `scheduleAtFixedRate()` and `scheduleWithFixedDelay()`.

The behavior of these two methods is slightly different. `scheduleAtFixedRate()` will activate a new copy of the task on a fixed timetable (and it will do so whether or not previous copies have completed), whereas `scheduleWithFixedDelay()` will activate a new copy of the task only after the previous instance has completed and the specified delay has elapsed.

Apart from the `ScheduledThreadPoolExecutor`, all the other pools we've met are obtained by choosing slightly different parameter choices for the quite general `ThreadPoolExecutor` class. For example, let's look at the following definition of `Executors.newFixedThreadPool()`:

```
public static ExecutorService newFixedThreadPool(int nThreads) {
    return new ThreadPoolExecutor(nThreads, nThreads,
                                  0L, TimeUnit.MILLISECONDS,
                                  new LinkedBlockingQueue<Runnable>());
}
```

This is the purpose of the helper methods, of course: to provide a convenient way of accessing some standard choices for a threadpool without needing to engage with the full complexity of `ThreadPoolExecutor`. Beyond the JDK, many other examples of executors and related threadpools exist, such as the `org.apache.catalina.Executor` class from the Tomcat web server.

Summary

- `java.util.concurrent` classes should be your preferred toolkit for all new multi-threaded Java code:
  - Atomic integers
  - Concurrent data structures, especially `ConcurrentHashMap`
  - Blocking queues and latches
  - Threadpools and executors

- These classes can be used to implement safe concurrent programming techniques including:
  - Addressing inflexibility of `synchronized` locks
  - Use of blocking queues for task handoff
  - Using latches for consensus among a group of threads
  - Partitioning execution into work units
  - Job control, including safe shutdown

Previous chapter

< 5 Java concurrency fundamentals

Next chapter

7 Understanding Java performance >